

| FORMULARIO DE PRODUCCIÓN DE ACTIVIDADES                          |
|------------------------------------------------------------------|
| ASIGNATURA: Algoritmos y Estructuras de Datos (Ciencia de Datos) |
| DOCENTE AUTOR: Miguel Quiroz Martinez                            |
| CORREO: mquiroz@ups.edu.ec                                       |

## ACTIVIDAD DE DESARROLLO — UNIDAD 3: GRAFOS

**Actividad:** Red de ciudades en Python: modelado, recorridos y ruta mínima

**Modalidad:** Deber de desarrollo individual, con implementación en Python y entrega del código en un repositorio público de GitHub.

### Descripción de la actividad:

Usted desarrollará en Python un programa de consola sencillo que modele una red de ciudades conectadas por carreteras y responda preguntas sobre ella usando las estructuras y algoritmos de la Unidad 3. Trabjará con un grafo ponderado propio de al menos 6 ciudades. Use las estructuras nativas de Python (diccionarios y listas); no es necesario usar bibliotecas externas como NetworkX ni optimizar más allá de lo visto en clase. Para el algoritmo de Dijkstra se le entrega el código ya implementado al final del enunciado: su tarea es integrarlo e interpretar sus resultados, no programarlo desde cero.

### Requerimientos funcionales (cada uno corresponde a una clase de la unidad):

- **R1 – Modelado del grafo.** Represente la red con una lista de adyacencia: un diccionario donde la clave es la ciudad y el valor es la lista de tuplas (ciudad\_vecina, distancia\_en\_km). Muestre el grado de cada ciudad, es decir, con cuántas ciudades conecta directamente.
- **R2 – Recorrido del grafo.** Implemente un BFS que, desde una ciudad de origen, liste todas las ciudades alcanzables y a cuántas escalas está cada una. Indique si la red está completamente conectada o si alguna ciudad queda aislada.
- **R3 – Ruta más barata con Dijkstra.** El algoritmo de Dijkstra se le entrega ya **implementado** al final de este enunciado: no debe programarlo desde cero. Su tarea es

**integrarlo** a su programa, ejecutarlo sobre su red desde una ciudad de origen e **interpretar el resultado**: muestre la distancia mínima en kilómetros hasta cada ciudad y señale cuál es la más lejana y cuál la más cercana.

- **R4 – Comparación e interpretación.** Elija dos ciudades y compare la ruta con menos escalas (según su BFS del R2) con la ruta de menor distancia en kilómetros (según Dijkstra del R3). Diseñe su red de modo que ambas rutas NO coincidan, y explique en el informe por qué ocurre.

### **Requerimientos técnicos y de análisis:**

- Organice el programa en un archivo principal (main.py) con un menú de consola sencillo, y puede separar funciones auxiliares en un segundo archivo (utilidades.py).
- Para la operación principal de cada requerimiento, incluya en el código un comentario con su complejidad en notación  $O$  (por ejemplo: `# BFS:  $O(V + E)$` ) y, en el informe, una breve tabla con esas complejidades.
- Incluya un archivo README.md en el repositorio que explique cómo ejecutar el programa y describa brevemente qué hace cada requerimiento.
- El código debe ejecutarse sin errores con `python main.py` e incluir la red de ciudades de ejemplo ya cargada para probarlo.

### **Entrega en GitHub:**

- Cree un repositorio público en GitHub con el nombre apellido\_nombre\_AlgEstDatos\_U3.
- Realice al menos 3 commits con mensajes descriptivos que evidencien el avance del trabajo.
- Entregue en la plataforma un documento PDF (máx. 2 páginas, Times New Roman o Calibri Light 12, interlineado 1,5) con: el enlace al repositorio, un dibujo o esquema de su red de ciudades, la tabla de complejidades y la explicación del R4. El enlace debe tener el formato `https://github.com/usuario/apellido_nombre_AlgEstDatos_U3`.

### **Bibliografía:**

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4.<sup>a</sup> ed.). MIT Press.

Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4.<sup>a</sup> ed.). Addison-Wesley Professional.

**Nombre del fichero (PDF):** “primerApellido\_primerNombre\_AlgEstDatos\_U3”, ejemplo: Lopez\_Juan\_AlgEstDatos\_U3.

**Formato de archivo a subir:** PDF (informe) y enlace al repositorio público de GitHub (código).

### Código provisto para el R3 (Dijkstra):

Copie esta función en su proyecto tal como está. Recibe el grafo en el formato del R1 (diccionario de listas de tuplas) y la ciudad de origen, y devuelve un diccionario con la distancia mínima hasta cada ciudad. No necesita modificarla: solo llamarla e interpretar lo que devuelve.

```
import heapq

def dijkstra(grafo, origen):
    """Devuelve {ciudad: distancia_minima_desde_origen}."""
    dist = {ciudad: float('inf') for ciudad in grafo}
    dist[origen] = 0
    heap = [(0, origen)]          # cola de prioridad
    while heap:
        d, u = heapq.heappop(heap) # el mas cercano pendiente
        if d > dist[u]:
            continue              # ya teniamos una ruta mejor
        for vecino, peso in grafo[u]:
            if dist[u] + peso < dist[vecino]: # relajacion
                dist[vecino] = dist[u] + peso
                heapq.heappush(heap, (dist[vecino], vecino))
    return dist

# Ejemplo de uso:
# distancias = dijkstra(mi_red, 'Quito')
# print(distancias) # {'Quito': 0, 'Ambato': 130, ...}
```

*Nota: la complejidad de esta función es  $O((V + E) \log V)$ . Debe reportarla en su tabla del informe y explicar, en una frase, de dónde sale el factor logarítmico.*

### Rúbrica de desarrollo (total: 9 puntos):

| Criterios                                      | Nivel Bajo                            | Nivel Medio                                       | Nivel Alto                                       | Pts |
|------------------------------------------------|---------------------------------------|---------------------------------------------------|--------------------------------------------------|-----|
| Implementación de los 4 requerimientos (R1–R4) | Resuelve 1 o ningún requerimiento (0) | Resuelve 2 o 3 requerimientos correctamente (1-2) | Los 4 requerimientos correctos y funcionales (3) | 3   |

|                                                                                              |                                                    |                                                       |                                                                                          |          |
|----------------------------------------------------------------------------------------------|----------------------------------------------------|-------------------------------------------------------|------------------------------------------------------------------------------------------|----------|
| Uso correcto del grafo y sus algoritmos (lista de adyacencia, BFS propio, Dijkstra provisto) | Uso incorrecto de las estructuras o algoritmos (0) | Uso correcto con errores menores (1)                  | Modela y recorre el grafo, e integra e interpreta correctamente el Dijkstra provisto (2) | 2        |
| Análisis de eficiencia: notación O en código e informe                                       | Ausente o incorrecto (0)                           | Presente con imprecisiones (0,5)                      | Correcto, comentado y justificado en la tabla (1)                                        | 1        |
| Uso correcto de GitHub: repositorio, $\geq 3$ commits descriptivos y README                  | Sin repositorio o sin commits útiles (0)           | Repositorio con pocos commits o README incompleto (1) | Repositorio público, $\geq 3$ commits descriptivos y README completo (2)                 | 2        |
| Informe PDF: esquema de la red, explicación del R4, formato y extensión                      | Ausente o sin justificación (0)                    | Explicación superficial o con fallos de formato (0,5) | Explica por qué la ruta más corta no es la de menos escalas (1)                          | 1        |
| <b>Totales</b>                                                                               |                                                    |                                                       |                                                                                          | <b>9</b> |

|                   |  |      |   |       |  |      |
|-------------------|--|------|---|-------|--|------|
| <b>DIFICULTAD</b> |  | Baja | x | Media |  | Alta |
|-------------------|--|------|---|-------|--|------|

|                               |   |                                                                         |
|-------------------------------|---|-------------------------------------------------------------------------|
| <b>COMPETENCIAS EVALUADAS</b> |   | Competencia 1: Dominio de estructuras de datos lineales y jerárquicas   |
|                               |   | Competencia 2: Implementación de diccionarios y estructuras asociativas |
|                               | x | Competencia 3: Modelado e implementación de grafos                      |
|                               | x | Competencia 4: Análisis de la eficiencia de algoritmos                  |
|                               |   | Competencia 5: Aplicación de técnicas de diseño de algoritmos           |

|                                 |   |                                                             |
|---------------------------------|---|-------------------------------------------------------------|
| <b>CONTENIDOS<br/>EVALUADOS</b> | x | Clase 7 / Fundamentos de Grafos                             |
|                                 | x | Clase 8 / Recorridos BFS y DFS                              |
|                                 | x | Clase 9 / Caminos Más Cortos y Árbol de Expansión<br>Mínima |

*Recuerde colocar la calificación de la tarea. Calificación: 9 puntos. Esta actividad evalúa las tres clases de la Unidad 3 (fundamentos de grafos, recorridos BFS y DFS, y caminos mínimos) e integra las competencias C3 (modelado e implementación de grafos) y C4 (análisis de eficiencia).*